

Computer Science

Coding Foundations

References vs Pointers in C++

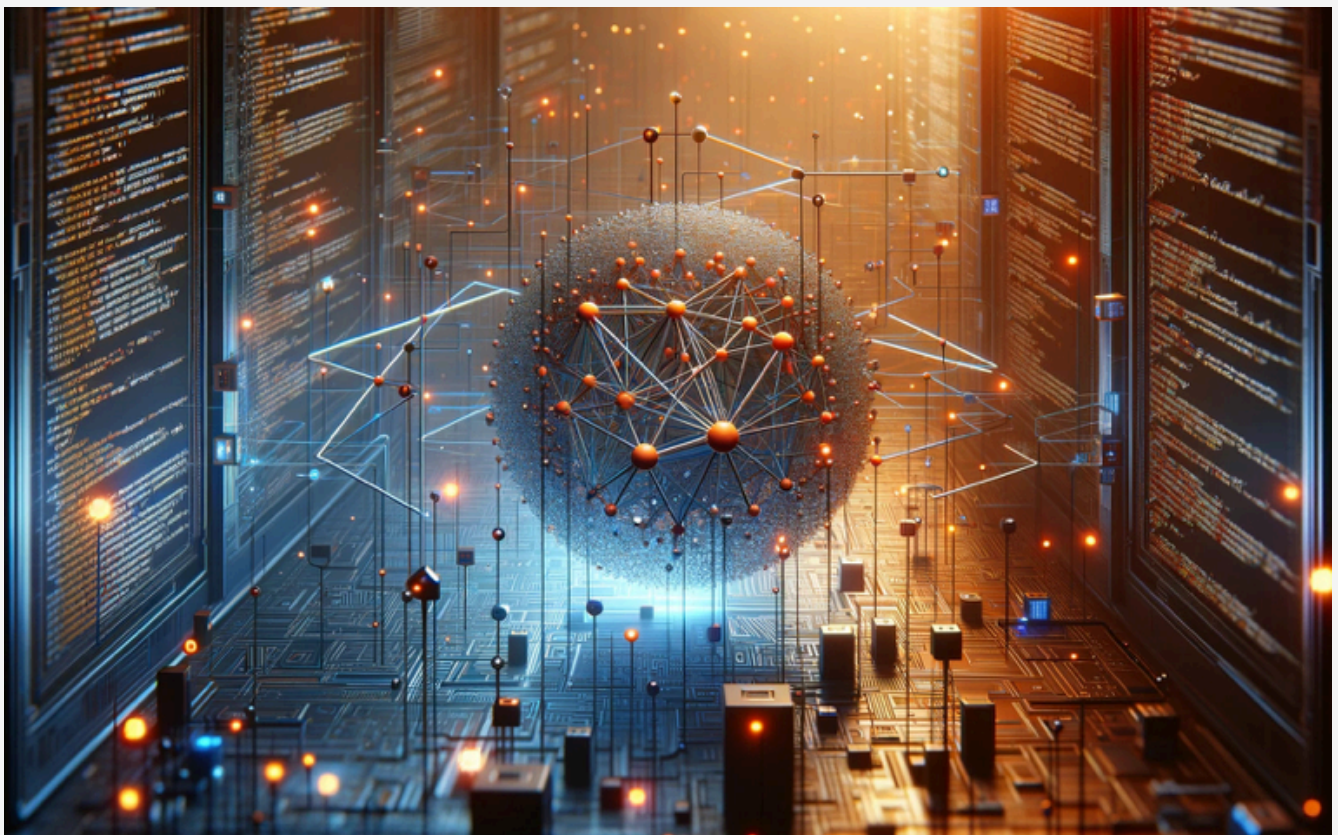
References and Pointers

by **Andrii Chorny**

Data Scientist, ML Engineer

Mar, 2024 •

9 min read



Introduction

In C++, both references and pointers provide ways to indirectly access variables. While they may seem similar at first glance, they serve different purposes and have distinct behaviors. Understanding the differences between references and pointers is crucial for effective C++ programming as it influences memory

management, function parameter passing, and object manipulation.

What are Pointers?

Pointers are variables that store memory addresses of other variables. They are a fundamental aspect of C++, allowing for dynamic memory management and the manipulation of arrays, functions, and structures.



• **Pointer Arithmetic:** Supports operations like increment (`++`) and decrement (`--`), enabling traversal through arrays.

- **Indirection:** The dereference operator (`*`) is used to access or modify the value at the memory address a pointer holds.
- **Nullability:** Pointers can be null (`nullptr` in C++11 and beyond), indicating they do not point to any memory location.

Example Usage:

```
1 int var = 10;  
2 int* ptr = &var; // ptr holds the address of var  
3 *ptr = 20; // Modifies var indirectly through ptr
```

Run Code from Your Browser - No Installation Required

Get started



What are References?

References in C++ act as an alias for another variable. Introduced to provide an alternative to pointers, references allow for cleaner syntax and safer code in certain scenarios.

Key Characteristics of References:

- **Alias:** A reference must be initialized upon declaration, creating a permanent association with the original variable.
- **No Reassignment:** Once established, a reference cannot be made to refer to a different variable.
- **No Null Reference:** References must always refer to an object or variable, avoiding the pitfalls of null pointers.
- **Implicit Dereferencing:** Unlike pointers, references do not require a dereference operator to access the value of the referenced variable.

Example Usage:

```
1 int var = 10;  
2 int& ref = var; // ref is an alias for var  
3 ref = 20; // Directly modifies var
```

Function Example

Here's a simple C++ example that demonstrates how both references and pointers can be used within a function to modify the value of variables passed to it. The example includes a function that increments the value of its parameters — one by using a pointer and the other by using a reference.

```

1 #include <iostream>
2
3 // Function to increment the value pointed to by a pointer
4 void incrementWithPointer(int* ptr) {
5     if (ptr) { // Check if the pointer is not null
6         (*ptr)++; // Dereference the pointer to increment the va
7     }
8 }
9
10 // Function to increment the value referred to by a reference
11 void incrementWithReference(int& ref) {
12     ref++; // Directly increment the referenced value
13 }
14
15 int main() {
16     int a = 5;
17     int b = 10;
18
19     // Incrementing 'a' using a pointer
20     incrementWithPointer(&a);
21
22     // Incrementing 'b' using a reference
23     incrementWithReference(b);
24
25     std::cout << "Values after incrementation: \n";
26     std::cout << "a = " << a << " (incremented by pointer)\n";
27     std::cout << "b = " << b << " (incremented by reference)\n";
28
29     return 0;
30 }

```

In this example:

- The `incrementWithPointer` function takes a pointer to an `int` as its parameter. It checks if the pointer is not null to avoid dereferencing a null pointer, then increments the value it points to.
- The `incrementWithReference` function takes a reference to an `int` as its parameter and increments the value directly. There's no need to check for nullity with references since they must refer to a valid object.
- In the `main` function, `a` is incremented using a pointer, and `b` is incremented using a reference. This demonstrates how the original values of `a` and `b` are modified through the use of pointers and references within functions.

Comparing References and Pointers

Syntax and Ease of Use

- **Pointers:** Require explicit dereferencing and address-of operations, making the syntax more complex but also more flexible.
- **References:** Offer simpler syntax, as they behave like normal variables and do not require dereferencing.

Memory Address Operations

- **Pointers:** Can be manipulated through arithmetic and can explicitly handle memory addresses.
- **References:** Hide the complexity of memory addresses, providing no direct way for arithmetic or memory address manipulation.

Function Parameters

- **Pointers:** Useful for passing large structures or arrays to functions without copying, with explicit syntax indicating indirection.
- **References:** Commonly used to pass objects to functions more cleanly, especially when modification of the original object is required.

Safety and Nullability

- **Pointers:** Prone to errors like null dereference, dangling pointers, and memory leaks if not carefully managed.

- **References:** Safer in the context of avoiding null and dangling references but require careful design to avoid unintended modifications.

**Start Learning Coding today
and boost your Career
Potential**

Start today!



Conclusion

In C++, both pointers and references have their place. Pointers offer powerful capabilities for dynamic memory management and low-level system programming, while references provide a safer and cleaner way to alias variables and pass parameters to functions. Understanding when to use each can significantly impact the clarity, safety, and efficiency of your C++ code.

FAQs

Q: Can a reference be reseated to refer to a different variable after its initialization?

A: No, once a reference is initialized to refer to a variable, it cannot be reseated to refer to a different variable. The reference will always alias the initially assigned variable.

Q: Is it possible to have a reference to a pointer and a pointer to a reference?

A: You can have a pointer to a pointer and a reference to a pointer, but not a pointer to a reference. References are not objects themselves; they are just aliases to existing objects, so they don't have a memory address that a pointer can hold. However, you can have a reference to a pointer, which is useful when you want to modify the pointer itself in a function.

Q: How can you return multiple values from a function using pointers and references?

A: You can return multiple values from a function by passing additional arguments as pointers or references. By modifying the values of these arguments inside the function, you can effectively return multiple results. This technique is commonly used when a function needs to output more than one piece of data.

Q: What happens if you try to dereference a null pointer?

A: Dereferencing a null pointer results in undefined behavior, which is a critical programming error. It can lead to crashes, memory corruption, or other unpredictable behavior. Always check if a pointer is null before dereferencing it.

Q: Are references more efficient than pointers?

A: References can be seen as more syntactically efficient than pointers because they provide a simpler and cleaner way to access variables without requiring explicit dereferencing. However, in terms of computational efficiency, references and pointers are generally equivalent. The compiler often implements references

under the hood as pointers, so the performance difference between them is negligible.

Q: How do references and pointers behave differently when used as function parameters?

A: When used as function parameters, pointers need to be dereferenced explicitly to access or modify the value they point to, whereas references allow direct access to the variable they refer to, making the code cleaner and easier to read. Additionally, since references cannot be null, using them as function parameters can implicitly signal that the argument must not be null, enhancing code safety.

Related courses

[See All Courses](#)

COURSE

Beginner

C++ Introduction

Start your path to becoming a skilled developer by mastering the foundational...



C++ | ★★★★★ 4.3

COURSE

Intermediate

C++ Pointers and References

Unlock the Power of Memory Manipulation. Dive deep into the fundamentals of...



C++ | ★★★★★ 4.4

Coding Foundations

Computer Science

BackEnd Development

The SOLID Principles in Software Development

The SOLID Principles Overview

by **Anastasiia Tsurkan**

Backend Developer

Nov, 2023 • 8 min read

Data Analytics

BackEnd Development

Computer Science

Coding Foundations

30 Python Project Ideas for Beginners

Python Project Ideas

by **Anastasiia Tsurkan**

Backend Developer

Sep, 2024 • 14 min read

Coding Foundations

Computer Science

Asynchronous Programming in Python

Brief Intro to Asynchronous Programming

by **Ruslan Shudra**

Data Scientist

Dec, 2023 • 5 min read

Topics

Technologies

Artificial Intelligence	Python	Dart
Data Visualization	SQL	ChatGPT
Data Science	HTML/CSS	Excel
Design	C++	Git
Machine Learning	R	
Web Development	JavaScript	
Development Tools	Java	
Marketing	React	
Frontend Development	C	
Backend Development	Golang	
Data Analytics	C#	
Game Development		

Learning Tracks

SQL from Zero to Hero	Frontend Development Foundation
Python Data Analysis and Visualization	C++ for Beginners
Python from Zero to Hero	Python: Beyond the Basics

Hero

Intermediate

Foundations of Machine
Learning

Java Essentials

Full Stack Web
Development

Become a React
Developer

Practical projects Plans

Beginner

Paid memberships

Intermediate

For Teams

Advanced

Company

Support

Help Center

Fraud Alert

Reviews

Blog

About

Contact us

Follow us



Address

Ucode Limited

Florinis 7, Greg Tower, 2nd Floor, 1065, Nicosia, Cyprus



[Terms & conditions](#)

[Privacy policy](#)

[Cookie policy](#)

[Money-Back Policy](#)

[Subscription terms](#)

Copyright 2026